

ReciApp

Sistema de Gestión de Reciclaje y Reducción de Emisiones CO2

Índice

Índice.....	2
1. ANÁLISIS DEL PROBLEMA.....	4
1.1. Introducción.....	4
1.2. Objetivos principales.....	4
1.3. Funciones y rendimientos deseados.....	5
1.4. Planteamiento y evaluación de diversas soluciones.....	6
1.5. Justificación de la solución elegida.....	7
1.6. Modelado de la solución.....	8
1.6.1. Recursos humanos.....	8
1.6.2. Recursos hardware.....	9
1.6.3. Recursos software.....	9
1.7. Planificación temporal.....	10
2. DISEÑO E IMPLEMENTACIÓN DEL PROYECTO.....	11
2.1. Plan de empresa.....	11
2.2. Arquitectura de la aplicación.....	12
2.3. Roles de usuario.....	15
2.4. Mapa de navegación.....	18
2.5. Base de datos.....	19
2.5.1. Diagrama Entidad-Relación.....	19
2.5.2. Paso a tablas.....	21
2.5.3. Datos de ejemplo.....	23
2.5.4. Script SQL de creación.....	24
2.6. Implementación por módulos.....	25
2.6.1. Acceso a Datos.....	25
2.6.2. Desarrollo de Interfaces.....	26
2.6.3. Programación de Servicios y Procesos.....	29
2.6.4. Programación Multimedia y Dispositivos Móviles.....	31
2.6.5. Terminal de Reciclaje (Python).....	33
3. FASE DE PRUEBAS.....	35
3.1. Pruebas de la API REST.....	35
3.2. Pruebas de la aplicación móvil.....	36
3.3. Pruebas de la aplicación de escritorio.....	38
3.4. Pruebas del terminal Python.....	39
3.5. Pruebas de integración.....	40
4. DOCUMENTACIÓN DE LA APLICACIÓN.....	42
4.1. Introducción a la aplicación.....	42

4.2. Manual de instalación.....	42
4.2.1. Requisitos previos.....	42
4.2.3. Instalación de la API REST.....	43
4.2.4. Instalación de la app móvil.....	43
4.2.5. Instalación de la app de escritorio.....	44
4.2.6. Instalación del terminal Python.....	44
4.2.7. Despliegue con Docker.....	45
4.3. Manual de usuario.....	46
4.3.1. Registro e inicio de sesión.....	46
4.3.2. Consultar productos.....	47
4.3.3. Consultar historial de reciclaje.....	48
4.3.4. Gestionar perfil.....	48
4.3.5. Uso del TAP en el terminal de reciclaje.....	49
4.3.6. Modo offline (app móvil).....	49
4.4. Manual de administración.....	49
4.4.1. Acceso al panel de administración.....	49
4.4.2. Gestion de usuarios.....	49
4.4.3. Gestión de productos.....	50
4.4.4. Gestión de transacciones.....	50
5. CONCLUSIONES FINALES.....	51
5.1. Grado de cumplimiento de los objetivos.....	51
5.2. Propuesta de modificaciones y ampliaciones futuras...	51
6. BIBLIOGRAFÍA.....	53

1. ANÁLISIS DEL PROBLEMA

1.1. Introducción

ReciApp es un sistema integral de gestión de reciclaje desarrollado como proyecto intermodular del CFGS de Desarrollo de Aplicaciones Multiplataforma (DAM). El proyecto aborda la necesidad de fomentar el reciclaje entre la población mediante la gamificación y el seguimiento del impacto ambiental individual, medido en kilogramos de CO2 reducidos.

El sistema se compone de cuatro aplicaciones interconectadas: una API REST centralizada desarrollada con Spring Boot, una aplicación móvil multiplataforma construida con React Native y Expo, una aplicación de escritorio con interfaz gráfica desarrollada en JavaFX, y un script de terminal en Python pensado para puntos de reciclaje físicos. Todas las aplicaciones se comunican con la API REST, que a su vez gestiona la persistencia de datos en una base de datos MySQL.

El funcionamiento del sistema es el siguiente: cada producto reciclable registrado en el catálogo tiene asociado un valor de emisiones de CO2 reducibles. Cuando un usuario recicla un producto (identificado mediante su código de barras), dicho valor se acumula en su perfil personal, permitiéndole visualizar su contribución individual a la reducción de emisiones de gases de efecto invernadero.

1.2. Objetivos principales

Se han establecido los siguientes objetivos principales para el desarrollo del proyecto los cuales todos han sido realizados:

- Desarrollar un sistema multiplataforma completo que permita registrar, consultar y gestionar acciones de reciclaje de productos.
- Implementar un mecanismo de seguimiento de emisiones de CO2 reducidas por cada usuario, fomentando la concienciación medioambiental.

- Diseñar una arquitectura cliente-servidor robusta con una API REST central que permita la comunicación entre todos los clientes.
- Crear una aplicación móvil accesible y fácil de usar para consultar productos y visualizar el impacto ambiental personal.
- Desarrollar una aplicación de escritorio con funcionalidades de administración completas (CRUD de usuarios, productos y transacciones).
- Implementar un terminal de reciclaje para puntos físicos que utilice códigos de barras y un token personal (TAP) para identificar al usuario.
- Garantizar la seguridad del sistema mediante autenticación JWT y cifrado de contraseñas con BCrypt.
- Desplegar el sistema en un servidor en la nube (AWS EC2) para demostrar su viabilidad en un entorno real.

1.3. Funciones y rendimientos deseados

El sistema implementado ofrece los siguientes servicios:

Gestión de usuarios

- Registro y autenticación con credenciales (nombre de usuario y contraseña).
- Gestión de perfil personal: modificación de nombre, cambio de contraseña y eliminación de cuenta.
- Sistema de roles diferenciados: cliente (usuario estándar) y administrador.
- Generación de TAP (Token de Autenticación Personal): número único de 6 dígitos para identificación en terminales físicos.

Gestión de productos

- Catálogo de productos reciclables con información detallada: nombre, material, código de barras, emisiones reducibles e imagen.
- Búsqueda por nombre o código de barras y filtrado por tipo de material.
- Operaciones CRUD completas para administradores.

Registro de reciclaje

- Registro de acciones de reciclaje asociando un producto a un usuario con fecha y hora exactas.
- Acumulación automática de emisiones de CO2 reducidas en el perfil del usuario.
- Historial completo de reciclajes con estadísticas de impacto ambiental.

Funcionalidades adicionales

- Modo offline/invitado en la aplicación móvil con datos de ejemplo almacenados localmente.
- Interfaz de terminal con colores y formato visual para puntos de reciclaje.

Rendimiento esperado

Tiempo de respuesta de la API inferior a 500 ms para operaciones estándar.

- Soporte de múltiples conexiones simultáneas desde los distintos clientes.
- Disponibilidad 24/7 gracias al despliegue en servidor cloud.

1.4. Planteamiento y evaluación de diversas soluciones

Durante la fase de planificación se evaluaron distintas aproximaciones para cada componente del sistema:

Backend / API

Se consideraron tres alternativas principales para el backend: Node.js con Express, Django REST Framework (Python) y Spring Boot (Java). Node.js ofrece simplicidad pero menor tipado y robustez. Django era una opción viable con buena documentación pero menor familiaridad por parte del desarrollador. Spring Boot, por su parte, proporcionaba un ecosistema maduro con Spring Security, Spring Data JPA y una integración natural con las aplicaciones Java del proyecto.

Aplicación móvil

Se evaluaron Flutter (Dart), desarrollo nativo (Kotlin/Swift) y React Native (TypeScript). Flutter ofrece buen rendimiento pero requería aprender Dart. El desarrollo nativo implicaba mantener dos bases de código. React Native con Expo permitía desarrollo multiplataforma con un lenguaje ampliamente conocido (TypeScript) y un ecosistema de herramientas que agiliza el desarrollo.

Aplicación de escritorio

Se compararon Electron (JavaScript), .NET WPF (C#) y JavaFX (Java). Electron generaba aplicaciones pesadas y duplica tecnologías con la app móvil. WPF limitaba la plataforma a Windows. JavaFX era multiplataforma, se integraba con el backend Java y cumplía con los requisitos del módulo de Desarrollo de Interfaces.

Base de datos

Se evaluaron PostgreSQL, MySQL y MongoDB. PostgreSQL y MySQL eran ambas salidas; se seleccionó MySQL por su mayor implantación en entornos educativos y compatibilidad con las herramientas del ciclo (LAMP).

1.5. Justificación de la solución elegida.

Componente	Tecnología	Justificación
Backend	Spring Boot 3.4.4 (Java 21)	Ecosistema maduro, Spring Security para JWT, Spring Data JPA, compatibilidad con la app JavaFX.
App móvil	React Native 0.81.5 + Expo SDK 54	Desarrollo multiplataforma real (Android/iOS), TypeScript, hot reload, Expo Go para desarrollo rápido.
App escritorio	JavaFX 25 (Java 24)	Multiplataforma, interfaz gráfica rica con FXML/CSS,

		módulo de Desarrollo de Interfaces.
Terminal	Python 3 + requests	Rapidez de desarrollo, ideal para scripts de terminal, colores ANSI nativos.
Base de datos	MySQL 8.x (InnoDB)	Relacional, transaccional, amplia compatibilidad, requisito del módulo de Acceso a Datos.

Esta combinación de tecnologías cubre la totalidad de los módulos del ciclo DAM: Acceso a Datos (Spring Data JPA, MySQL, API REST), Desarrollo de Interfaces (JavaFX), Programación de Servicios y Procesos (comunicación HTTP, JWT, múltiples clientes simultáneos) y Programación Multimedia y Dispositivos Móviles (React Native con Expo).

1.6. Modelado de la solución

1.6.1. Recursos humanos

El proyecto ha sido desarrollado de forma individual por un único alumno del CFGS de DAM (Yo, José Aarón Sánchez Martín), asumiendo todos los roles del desarrollo: análisis, diseño de base de datos, desarrollo backend, desarrollo frontend (móvil y escritorio), scripting (terminal Python), despliegue en servidor y documentación.

Aunque al final de la documentación habrá una serie de agradecimientos a determinadas personas por su apoyo a lo largo del proyecto, ya sea como sencillamente apoyo emocional o por que hicieran algún tipo de labor que quiera agradecer.

1.6.2. Recursos hardware

Recurso	Especificaciones	Uso
Portátil para desarrollo	ASUS FA506NC Windows 11	Desarrollo de todas las aplicaciones y pruebas locales
Instancia EC2 (AWS)	t2.micro Ubuntu 22.04 LTS	Alojamiento de la API REST y MySQL en producción

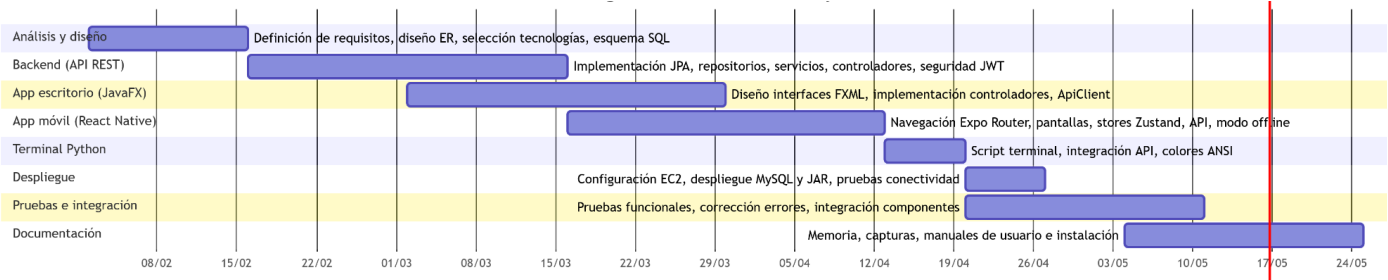
1.6.3. Recursos software

Software	Propósito
VS Code	IDE principal para Java y TypeScript
JDK 24 (Temurin)	Compilación del backend y la app de escritorio
Android Studio (Pixel_8_API_35)	Entorno de ejecución para Expo / React Native
Python 3.14.0	Ejecución del terminal de reciclaje
LAMP	Servidor MySQL y phpMyAdmin
Postman	Pruebas de la API REST
Git + GitHub	Control de versiones y repositorio remoto
Expo Go	Ejecución de la app móvil en dispositivo físico
Docker	Contenedorización
WixTools	Creación del instalador para windows
JDK 17 + CMake + NDK 27	Compilar a APK

1.7. Planificación temporal

Fase	Periodo	Tareas Principales
Análisis y diseño	Febrero 2026 (semanas 1-2)	Definición de requisitos, diseño del modelo ER, selección de tecnologías, creación del esquema SQL.
Backend (API REST)	Febrero - Marzo 2026 (semanas 3-6)	Implementación de entidades JPA, repositorios, servicios, controladores, seguridad JWT.
App de escritorio (JavaFX)	Marzo 2026 (semanas 5-8)	Diseño de interfaces FXML, implementación de controladores, ApiClient.
App mobile (React Native)	Marzo - Abril 2026 (semanas 7-10)	Navegación con Expo Router, pantallas, stores Zustand, servicio API, modo offline.
Terminal Python	Abril 2026 (semana 11)	Script de terminal, integración con API, interfaz con colores ANSI.
Despliegue	Abril 2026 (semana 12)	Configuración de EC2, despliegue de MySQL y JAR, pruebas de conectividad.
Pruebas e integración	Abril - Mayo 2026 (semanas 12-14)	Pruebas funcionales, corrección de errores, pruebas de integración entre componentes.

Documentación	Mayo 2026 (semanas 14-16)	Redacción de la memoria, capturas de pantalla, manuales de usuario e instalación.
---------------	---------------------------	---



2. DISEÑO E IMPLEMENTACIÓN DEL PROYECTO

2.1. Plan de empresa

ReciApp se enmarca en una empresa ficticia denominada EcoTech Solutions S.L., una startup tecnológica especializada en soluciones digitales para la sostenibilidad medioambiental.

Datos de la empresa

Campo	Valor
Nombre	EcoTech Solutions S.L.
Sector	Tecnología
Sede	Madrid, España
Actividad	Desarrollo de aplicaciones para la gestión del reciclaje
Plantilla	5 empleados (1 CEO, 2 desarrolladores, 1 diseñador UX, 1 sysadmin)
Clientes	Ayuntamientos, comunidades de vecinos, centros educativos,

	empresas con políticas ESG
--	----------------------------

Modelo de negocio

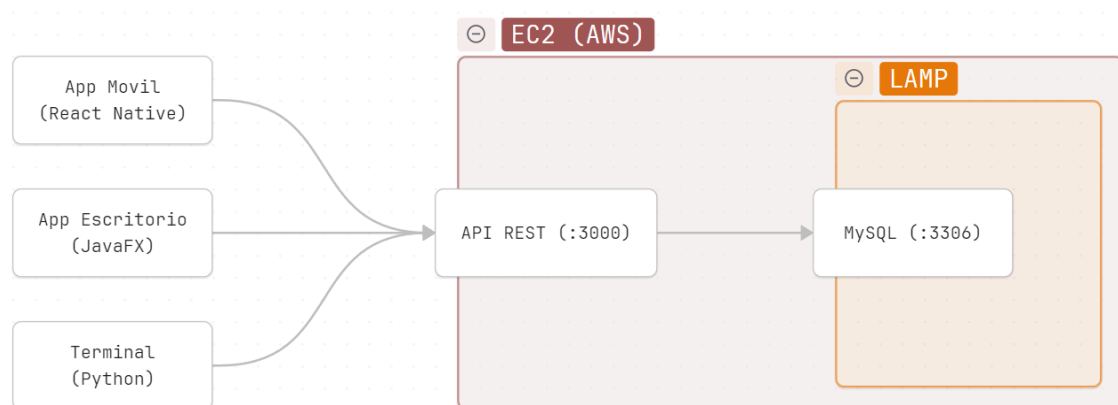
La empresa ofrece ReciApp como servicio (SaaS) a municipios y organizaciones que deseen implantar un sistema de reciclaje con trazabilidad. Los ingresos provienen de licencias mensuales por número de usuarios activos y de la instalación de terminales de reciclaje en puntos estratégicos.

Propuesta de valor

- Seguimiento individual del impacto ambiental de cada ciudadano/empleado.
- Gamificación: rankings, acumulación de CO2 reducido, TAP como identificación personal.
- Multiplataforma: accesible desde móvil, escritorio y terminales físicos.
- Datos abiertos para que las organizaciones midan su huella ecológica.

2.2. Arquitectura de la aplicación

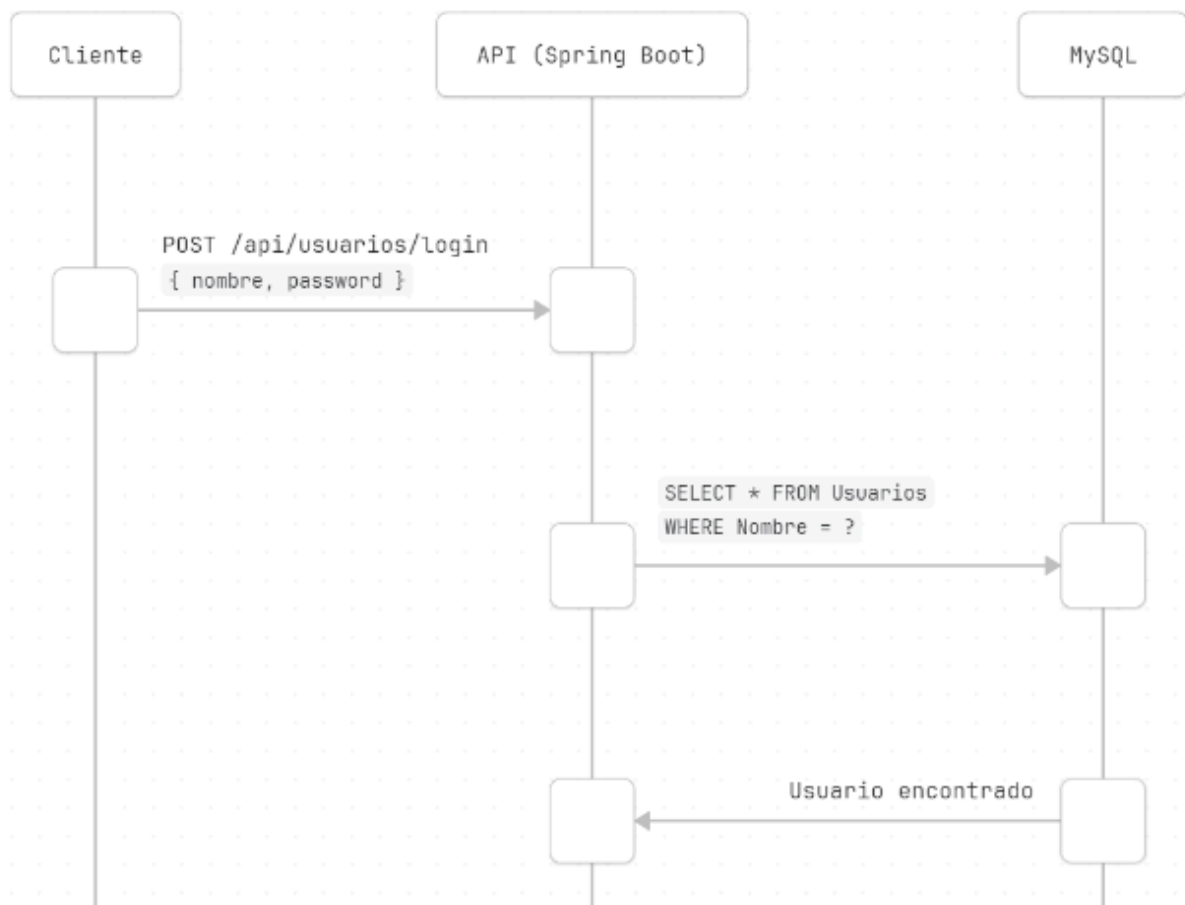
ReciApp sigue una arquitectura cliente-servidor de tres capas: presentación (los tres clientes), lógica de negocio (API REST con Spring Boot) y persistencia (MySQL). Los tres clientes se comunican exclusivamente con la API REST mediante peticiones HTTP en el puerto 3000, utilizando JSON como formato de intercambio de datos.

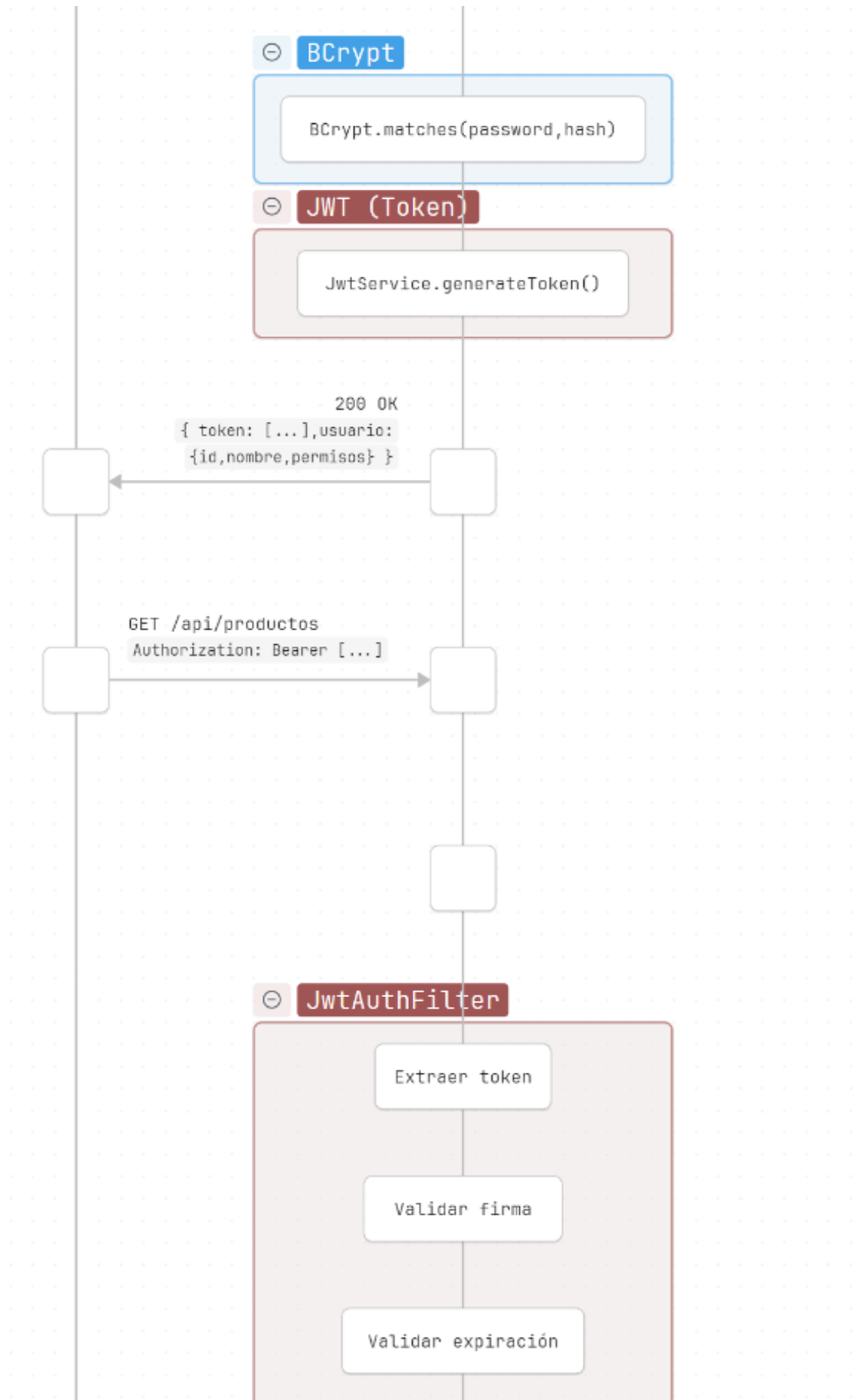


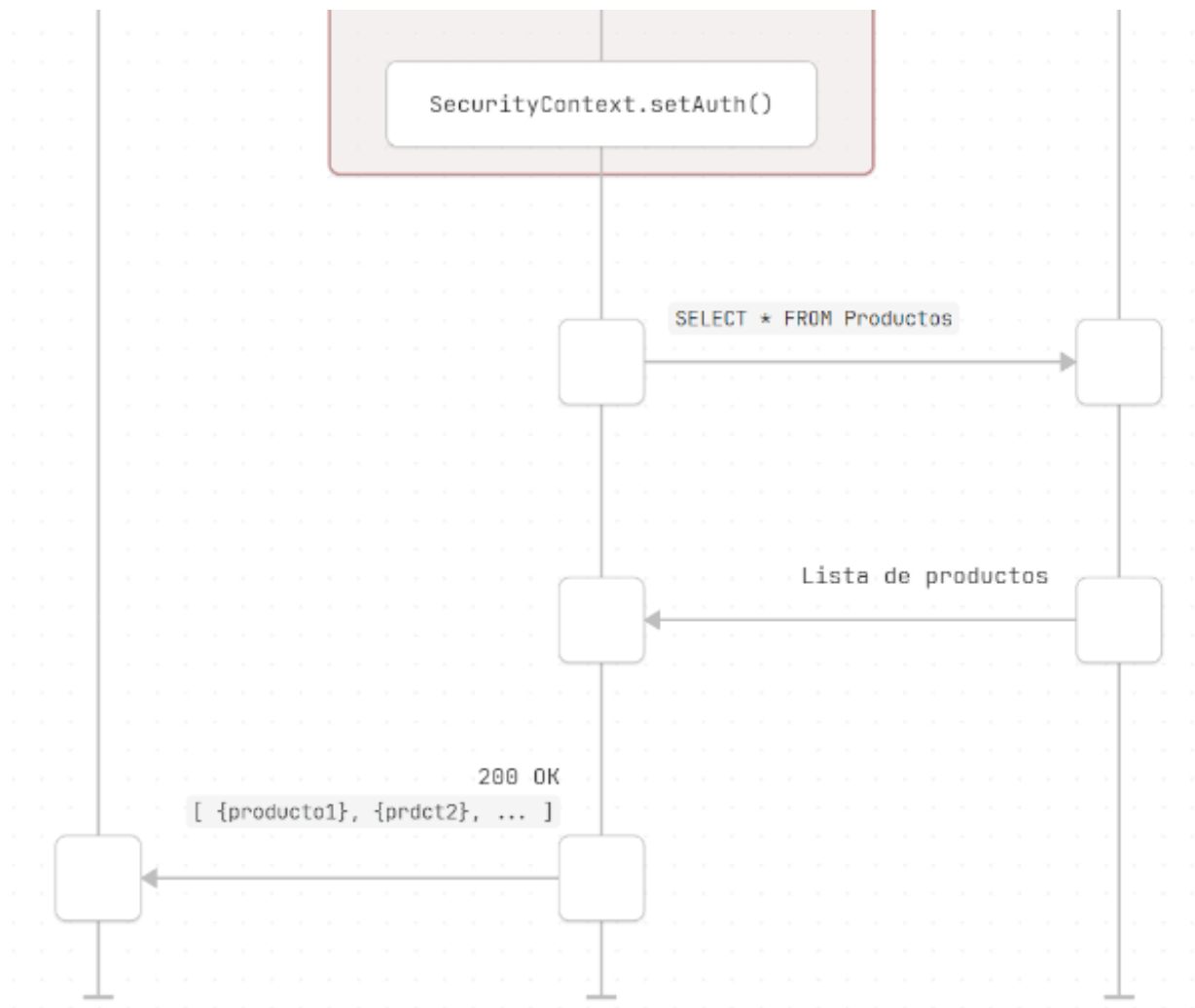
Flujo de autenticación (JWT)

El sistema utiliza JSON Web Tokens (JWT) para la autenticación stateless. El flujo es el siguiente:

- El cliente envía credenciales (nombre + contraseña) mediante POST a `/api/usuarios/login`.
- El servidor valida la contraseña con BCrypt (strength 10) y genera un token JWT firmado con HMAC-SHA256.
- El token contiene los claims: `userId`, nombre y permisos, con una validez de 7 días.
- En cada petición posterior, el cliente envía el token en la cabecera `Authorization: Bearer <token>`.
- El filtro `JwtAuthFilter` intercepta cada petición, valida el token y establece el `SecurityContext` de Spring.







2.3. Roles de usuario

El sistema define dos roles de usuario almacenados en la columna Permisos de la tabla Usuarios, más un modo invitado exclusivo de la aplicación móvil.

Rol: Cliente

Es el rol por defecto asignado a todo usuario nuevo al registrarse. Un usuario con rol cliente puede:

- Registrarse e iniciar sesión en cualquiera de las plataformas.
- Consultar el catálogo completo de productos reciclables.

- Visualizar su historial personal de reciclaje y emisiones acumuladas.
- Solicitar y renovar su TAP (Token de Autenticación Personal) de 6 dígitos.
- Modificar su nombre de usuario y contraseña.
- Eliminar su cuenta (requiere confirmar con contraseña).
- Reciclar productos en terminales físicos usando su TAP.

Rol: Administrador

Tiene todos los permisos del cliente, más las siguientes funcionalidades de administración:

- Gestión completa (CRUD) de usuarios a través de la API (/api/admin/usuarios) y de la app de escritorio.
- Gestión completa (CRUD) de productos: crear, listar, modificar y eliminar, incluyendo imágenes en Base64.
- Gestión completa (CRUD) de transacciones de reciclaje.
- Operación del terminal de reciclaje en puntos físicos.

Modo Invitado (solo app móvil)

La aplicación móvil permite un modo invitado para uso sin conexión o sin registro. El usuario invitado puede explorar el catálogo de productos con datos de ejemplo almacenados localmente, pero no tiene acceso a funcionalidades que requieran autenticación (historial, TAP, gestión de perfil). Se muestra un banner informativo en las pantallas con funcionalidad restringida.

Matriz de permisos

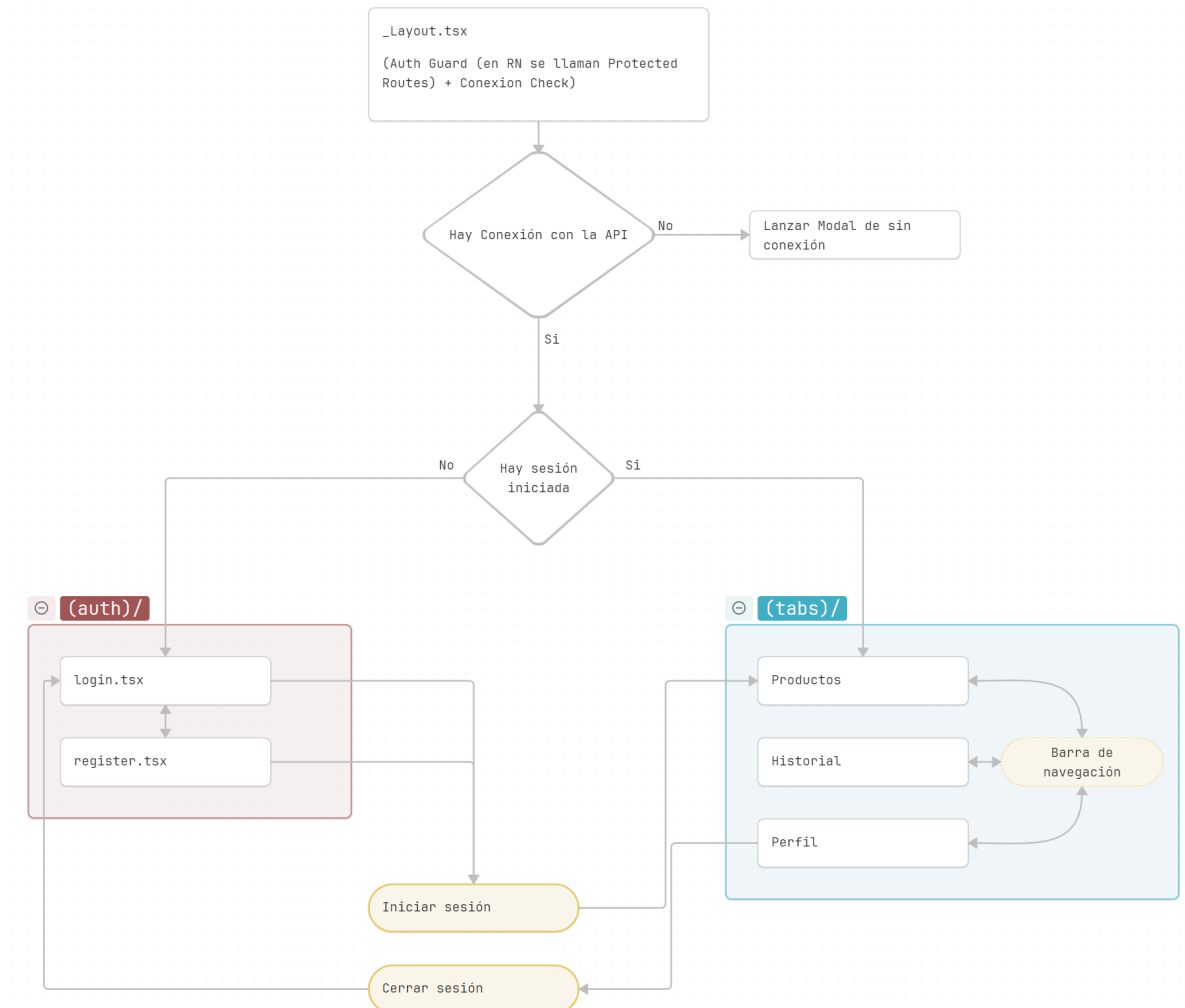
La Tabla 1 resume los permisos de cada rol en el sistema. La autorización se implementa tanto a nivel de API (anotaciones @PreAuthorize en Spring Security) como a nivel de interfaz (visibilidad condicional de elementos).

Acción	Cliente	Admin	Invitado
Registro / Login	Sí	Sí	No

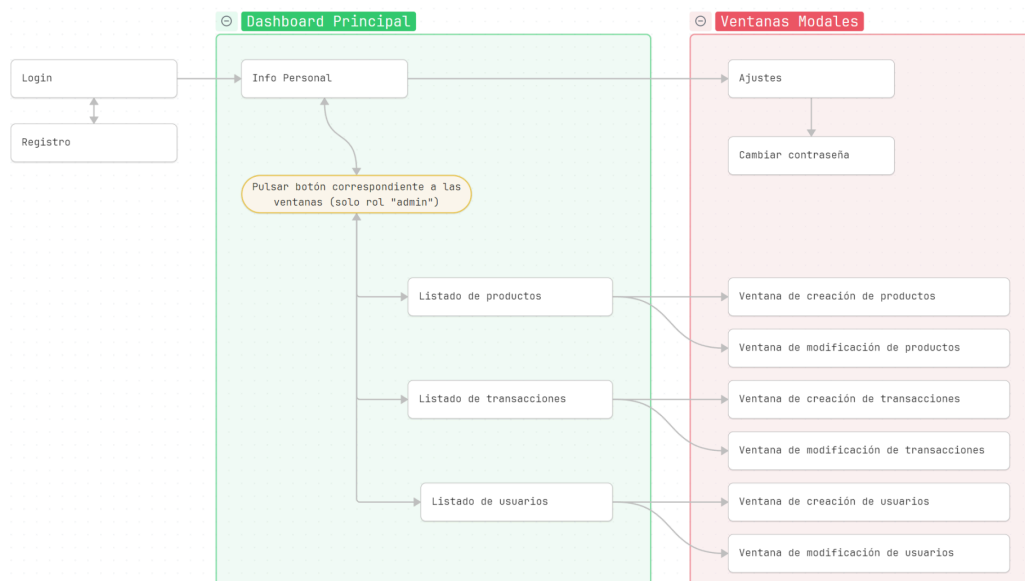
Ver catálogo de productos	Sí	Sí	Sí (ejemplo)
Ver historial propio	Sí	Sí	No
Ver / solicitar TAP	Sí	Sí	No
Modificar nombre / contraseña	Sí	Sí	No
Eliminar cuenta propia	Sí	Sí	No
CRUD Usuarios (admin)	No	Sí	No
CRUD Productos (admin)	No	Sí	No
CRUD Transacciones (admin)	No	Sí	No
Operar terminal de reciclaje	No	Sí	No

2.4. Mapa de navegación

App móvil (React Native / Expo)



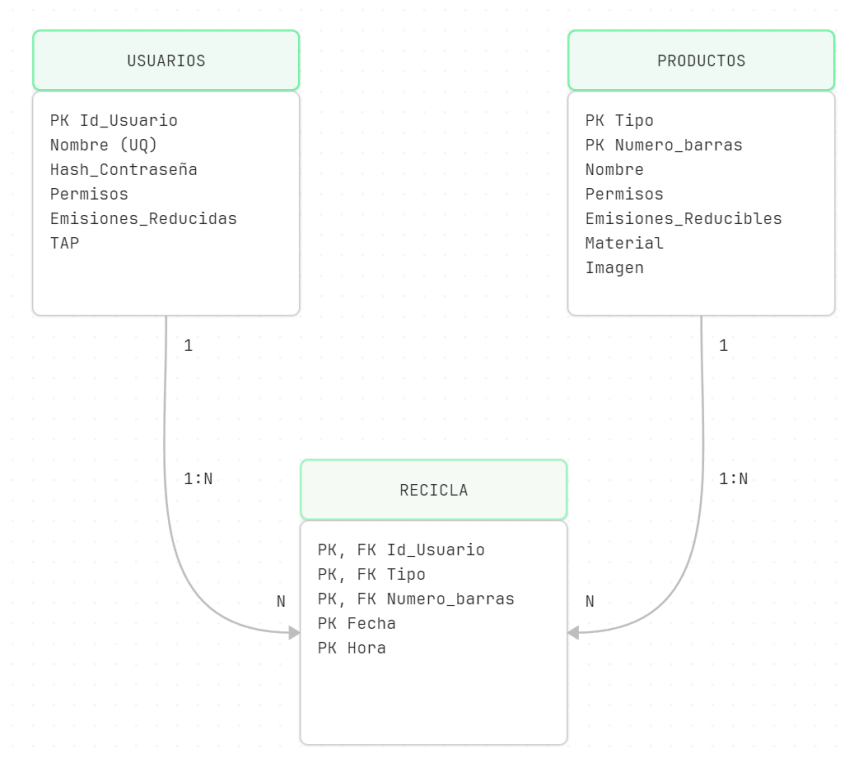
App de escritorio (JavaFX)



2.5. Base de datos

2.5.1. Diagrama Entidad-Relación

La base de datos `reciInventario_db` utiliza MySQL con el motor de almacenamiento InnoDB y codificación utf8mb4. El modelo consta de tres entidades principales con las siguientes relaciones:



Relaciones del modelo:

- Usuarios - Recicla: relación 1:N. Un usuario puede tener muchos registros de reciclaje.
- Productos - Recicla: relación 1:N. Un producto puede aparecer en muchos registros de reciclaje.
- Recicla: tabla intermedia con clave primaria compuesta de cinco columnas (Id_Usuario, Tipo, Numero Barras, Fecha, Hora), lo que permite registrar múltiples reciclajes del mismo producto por el mismo usuario en instantes diferentes.
- Ambas claves foráneas tienen ON DELETE CASCADE: al eliminar un usuario o producto, se eliminan automáticamente sus registros de reciclaje.

2.5.2. Paso a tablas

Tabla: Usuarios

Columna	Tipo de dato	Restricciones	Descripción
Id_Usuario	INT	PK, AUTO_INCREMENT	Identificador único del usuario
Nombre	VARCHAR(50)	NOT NULL, UNIQUE	Nombre de usuario para inicio de sesión
Hash_Contraseña	VARCHAR(100)	NOT NULL	Hash BCrypt de la contraseña (strength 10)
Permisos	VARCHAR(15)	DEFAULT 'cliente'	Rol del usuario: 'cliente' o 'administrador'
Emissiones_Reducidas	FLOAT	DEFAULT 0	Kilogramos de CO2 reducidos acumulados
TAP	INT	DEFAULT NULL	Token de Autenticación Personal (6 dígitos, único)

Tabla: Productos

Utiliza una clave primaria compuesta por el tipo de código de barras y el número de barras, lo que permite gestionar distintos formatos de códigos (EAN13, UPC, etc.).

Columna	Tipo de dato	Restricciones	Descripción
Tipo	VARCHAR(10)	PK (compuesta)	Formato del código de barras (p.ej. EAN13)
Numero_barras	BIGINT	PK (compuesta)	Número del código de barras
Nombre	VARCHAR(50)	--	Nombre comercial del producto
Emissiones_Reducibles	FLOAT	--	kg de CO2 que se reducen al reciclar el producto
Material	VARCHAR(15)	--	Material principal (PET, Vidrio, Aluminio, Brick...)
Imagen	LONGTEXT	--	Imagen del producto codificada en Base64

Tabla: Recicla (historial de reciclaje)

Registra cada acción de reciclaje. Su clave primaria compuesta de cinco columnas garantiza la unicidad temporal de cada registro.

Columna	Tipo de dato	Restricciones	Descripción
Id_Usuario	INT	PK, FK → Usuarios	Usuario que realizó el reciclaje
Tipo	VARCHAR (10)	PK, FK → Productos	Tipo de código del producto
Numero_barras	BIGINT	PK, FK → Productos	Código de barras del producto
Fecha	DATE	PK	Fecha en que se realizó el reciclaje
Hora	TIME	PK	Hora exacta del reciclaje

2.5.3. Datos de ejemplo

El script de inicialización (000_create_database.sql) incluye cinco productos de ejemplo para facilitar las pruebas del sistema:

Producto	Código EAN13	Material	CO2 reducible (kg)
Coca-Cola 330ml	8410076472885	Aluminio	0.033
Agua Bezoya 1.5L	8411700000017	PET	0.025
Leche Hacendado 1L	8480000291455	Brick	0.040
Mahou Clásica 330ml	8410128800505	Vidrio	0.045

Fanta Naranja 330ml	54490000 00996	Aluminio	0.033
---------------------	-------------------	----------	-------

2.5.4. Script SQL de creación

```
CREATE DATABASE IF NOT EXISTS `reciInventario_db`
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_unicode_ci;
```

```
USE `reciInventario_db`;
```

```
CREATE TABLE IF NOT EXISTS `Usuarios` (
  `Id_Usuario`      INT          NOT NULL AUTO_INCREMENT,
  `Nombre`          VARCHAR(50)  NOT NULL,
  `Hash_Contrasena` VARCHAR(100) NOT NULL,
  `Permisos`         VARCHAR(15)  DEFAULT 'cliente',
  `Emisiones_Reducidas` FLOAT      DEFAULT 0,
  `TAP`             INT          DEFAULT NULL,
  PRIMARY KEY (`Id_Usuario`),
  UNIQUE KEY `uk_nombre` (`Nombre`)
) ENGINE = InnoDB
  DEFAULT CHARSET = utf8mb4;
```

```
CREATE TABLE IF NOT EXISTS `Productos` (
  `Tipo`            VARCHAR(10)  NOT NULL,
  `Numero_barras`   BIGINT       NOT NULL,
  `Nombre`          VARCHAR(50)  DEFAULT NULL,
  `Emisiones_Reducibles` FLOAT    DEFAULT NULL,
  `Material`        VARCHAR(15)  DEFAULT NULL,
  `Imagen`          LONGTEXT     DEFAULT NULL,
  PRIMARY KEY (`Tipo`, `Numero_barras`)
) ENGINE = InnoDB
  DEFAULT CHARSET = utf8mb4;
```

```
CREATE TABLE IF NOT EXISTS `Recicla` (
  `Id_Usuario`      INT          NOT NULL,
  `Tipo`            VARCHAR(10)  NOT NULL,
  `Numero_barras`   BIGINT       NOT NULL,
  `Fecha`           DATE         NOT NULL,
```



```
    `Hora`          TIME      NOT NULL,  
    PRIMARY KEY (`Id_Usuario`, `Tipo`, `Numero_barras`, `Fecha`,  
    `Hora`),  
    CONSTRAINT `fk_recicla_usuario`  
        FOREIGN KEY (`Id_Usuario`)  
        REFERENCES `Usuarios` (`Id_Usuario`)  
        ON DELETE CASCADE,  
    CONSTRAINT `fk_recicla_producto`  
        FOREIGN KEY (`Tipo`, `Numero_barras`)  
        REFERENCES `Productos` (`Tipo`, `Numero_barras`)  
        ON DELETE CASCADE  
) ENGINE = InnoDB  
DEFAULT CHARSET = utf8mb4;
```

2.6. Implementación por módulos

A continuación se detalla cómo el proyecto cubre cada uno de los módulos del ciclo formativo de DAM.

2.6.1. Acceso a Datos

El módulo de Acceso a Datos se cubre principalmente a través de la API REST desarrollada con Spring Boot y Spring Data JPA:

- Utilización de un ORM (Hibernate/JPA): las entidades Java (Usuario, Producto, Recicla) se mapean directamente a las tablas de MySQL mediante anotaciones JPA (@Entity, @Table, @Id, @Column, @ManyToOne, @EmbeddedId).
- Repositorios Spring Data JPA: las interfaces UsuarioRepository, ProductoRepository y ReciclaRepository extienden JpaRepository, proporcionando operaciones CRUD automáticas y consultas derivadas del nombre del método (findByNombre, findByTap, existsByNombre).
- API REST como método de acceso preferente: todos los clientes acceden a la base de datos exclusivamente a través de la API REST (endpoints HTTP), cumpliendo con la recomendación del proyecto de utilizar una API REST como método más versátil y efectivo.
- Base de datos relacional remota: MySQL se ejecuta en el servidor EC2, siendo accedida por la API mediante JDBC (driver mysql-connector-j).

- DTOs para transferencia de datos: se utilizan objetos DTO (LoginRequest, AuthResponse, ProductoDto, HistorialDto, etc.) para desacoplar la capa de presentación de las entidades de persistencia.

Estructura de la capa de acceso a datos:

Carpeta/Archivo	Descripción
api-spring/src/main/java/com/reciapp/api/	
entity/	
Usuario.java	JPA @Entity -> tabla Usuarios
Producto.java	JPA @Entity -> tabla Productos
ProductoId.java	@EmbeddedId, clave compuesta
Recicla.java	JPA @Entity -> tabla Recicla
ReciclaId.java	@EmbeddedId, clave compuesta
repository/	
UsuarioRepository.java	extends JpaRepository
ProductoRepository.java	extends JpaRepository
ReciclaRepository.java	extends JpaRepository
service/	
UsuarioService.java	logica de negocio usuarios
ProductoService.java	logica de negocio productos
ReciclaService.java	logica de negocio reciclaje
AdminService.java	operaciones de administracion

2.6.2. Desarrollo de Interfaces

El módulo de Desarrollo de Interfaces se cubre con la aplicación de escritorio desarrollada en JavaFX 25:

- Interfaz gráfica multiplataforma con JavaFX y FXML: la aplicación cuenta con 12 vistas FXML que definen las interfaces de forma declarativa, separando la estructura visual (FXML) de la lógica (controladores Java).
- Estilización mediante CSS: se utiliza un archivo styles.css para personalizar la apariencia de los componentes JavaFX.
- Formularios para gestión de datos: se han implementado formularios completos para crear y modificar usuarios, productos y transacciones, con validación de campos.
- Interfaz de administración: pestañas para gestión de información personal, usuarios, productos y transacciones con formularios completos.

- Navegación entre ventanas: la clase StartWin gestiona la transición entre pantallas mediante carga dinámica de archivos FXML.

Vistas FXML implementadas:

Vista FXML	Controlador	Función
loginStart_win.fxml	LoginController	Inicio de sesión
singUp_win.fxml	SingUpController	Registro de nuevo usuario
main_win.fxml	MainController	Dashboard con 2 pestañas (info personal, admin)
settings_win.fxml	SettingsController	Configuración del usuario
changePasswd.fxml	ChangePasswd	Cambio de contraseña
escanear_win.fxml	Escanear	Escaneo de códigos de barras
newProducto.fxml	NewProducto	Crear nuevo producto
modProducto.fxml	ModProducto	Modificar producto existente
newUser.fxml	NewUser	Crear nuevo usuario
modUser.fxml	ModUser	Modificar usuario existente
newTransaccion.fxml	NewTransaccion	Crear nueva transacción
modTransaccion.fxml	ModTransaccion	Modificar transacción

Así se verían algunas de las capturas de pantalla.

Reci Inventario - Principal

Personal Usuarios Productos Transacciones Ajustes Salir

Personal Tablas

Nombre: adminb

Emisiones Reducidas: 202.8 kg CO2

Permisos: administrador

GENERAR TAP ESCANEAR

Mod

Nombre
botellín de agua

Código Barras
654634

Tipo
EAN13

Emisiones
0.1

Material
Plástico mixto

Imagen actual: si (29732 caracteres)
Nueva ruta (opcional) Copiar Ba...

Aceptar Cancelar

2.6.3. Programación de Servicios y Procesos

El módulo de Programación de Servicios y Procesos se cubre con los siguientes aspectos del proyecto:

- Comunicación en red mediante HTTP: todos los clientes se comunican con la API REST a través de peticiones HTTP (GET, POST, PUT, DELETE) utilizando sockets TCP de forma transparente.
- Servidor web concurrente: Spring Boot gestiona múltiples hilos de ejecución para atender peticiones simultáneas de los distintos clientes (móvil, escritorio y terminal).
- HttpClient multihilo en JavaFX: la aplicación de escritorio utiliza `java.net.http.HttpClient`, que internamente usa un pool de hilos para gestionar conexiones HTTP asíncronas.
- Peticiones asíncronas en React Native: la app móvil realiza peticiones HTTP asíncronas (`fetch/async-await`) sin bloquear el hilo principal de la interfaz de usuario.
- Sesiones HTTP en Python: el terminal utiliza `requests.Session()` para mantener la conexión y reutilizar cabeceras (incluyendo el token JWT) entre múltiples peticiones.

Endpoints de la API REST:

Endpoints públicos (sin autenticación):

Método	Endpoint	Descripción
GET	/api/health	Comprobación de estado del servidor
POST	/api/usuarios/login	Autenticación (devuelve JWT)
POST	/api/usuarios/register	Registro de nuevo usuario

Endpoints protegidos (requieren JWT):

Método	Endpoint	Descripción
GET	/api/usuarios/profile	Perfil del usuario autenticado
PUT	/api/usuarios/{id}/nombre	Actualizar nombre

PUT	/api/usuarios/{id}/password	Cambiar contraseña
PUT	/api/usuarios/{id}/tap	Solicitar/renovar TAP
DELETE	/api/usuarios/{id}	Eliminar cuenta
GET	/api/usuarios/by-tap/{tap}	Buscar usuario por TAP
GET	/api/productos	Listar todos los productos
GET	/api/productos/barcode/{code}	Buscar producto por código de barras
GET	/api/historial	Historial de reciclaje del usuario
POST	/api/historial	Registrar nuevo reciclaje

Endpoints de administración (requieren JWT + rol administrador):

Método	Endpoint	Descripción
GET	/api/admin/usuarios	Listar todos los usuarios
POST	/api/admin/usuarios	Crear usuario
PUT	/api/admin/usuarios/{id}	Modificar usuario
DELETE	/api/admin/usuarios/{id}	Eliminar usuario
GET	/api/admin/productos	Listar todos los productos
POST	/api/admin/productos	Crear producto
PUT	/api/admin/productos/{tipo}/{barras}	Modificar producto
DELETE	/api/admin/productos/{tipo}/{barras}	Eliminar producto

GET	/api/admin/transacciones	Listar transacciones
POST	/api/admin/transacciones	Crear transacción
DELETE	/api/admin/transacciones	Eliminar transacción

2.6.4. Programación Multimedia y Dispositivos Móviles

El módulo de Programación Multimedia y Dispositivos Móviles se cubre con la aplicación móvil desarrollada en React Native 0.81.5 con Expo SDK 54 y TypeScript:

- Interactividad del usuario: la app ofrece navegación por pestañas (Productos, Historial, Perfil), pull-to-refresh, búsqueda interactiva, filtros por material, modales, formularios y animaciones de transición.
- Intercambio de información: se comunica con la API REST para obtener y enviar datos en tiempo real, con soporte para modo offline con datos cacheados en AsyncStorage.
- Usabilidad: diseñada siguiendo principios de Material Design con NativeWind (Tailwind CSS) para un estilo coherente, Ionicons para iconografía y React Native Paper para componentes estándar.

Estructura de la app móvil:**app/**

- |— **_layout.tsx** <-- Layout raíz: auth guard + verificación de conexión
- |— **index.tsx** <-- Punto de entrada
- |— **connection-modal.tsx** <-- Modal de modo offline
- |— **(auth)/**
 - | |— **_layout.tsx**
 - | |— **login.tsx** <-- Pantalla de inicio de sesión
 - | |— **register.tsx** <-- Pantalla de registro
- |— **(tabs)/**
 - | |— **_layout.tsx** <-- Tab navigator (3 pestañas visibles)
 - | |— **productos/**
 - | | |— **index.tsx** <-- Catálogo de productos
 - | | |— **[barcode].tsx** <-- Detalle de producto
 - | |— **historial.tsx** <-- Historial de reciclaje
 - | |— **perfil.tsx** <-- Perfil del usuario

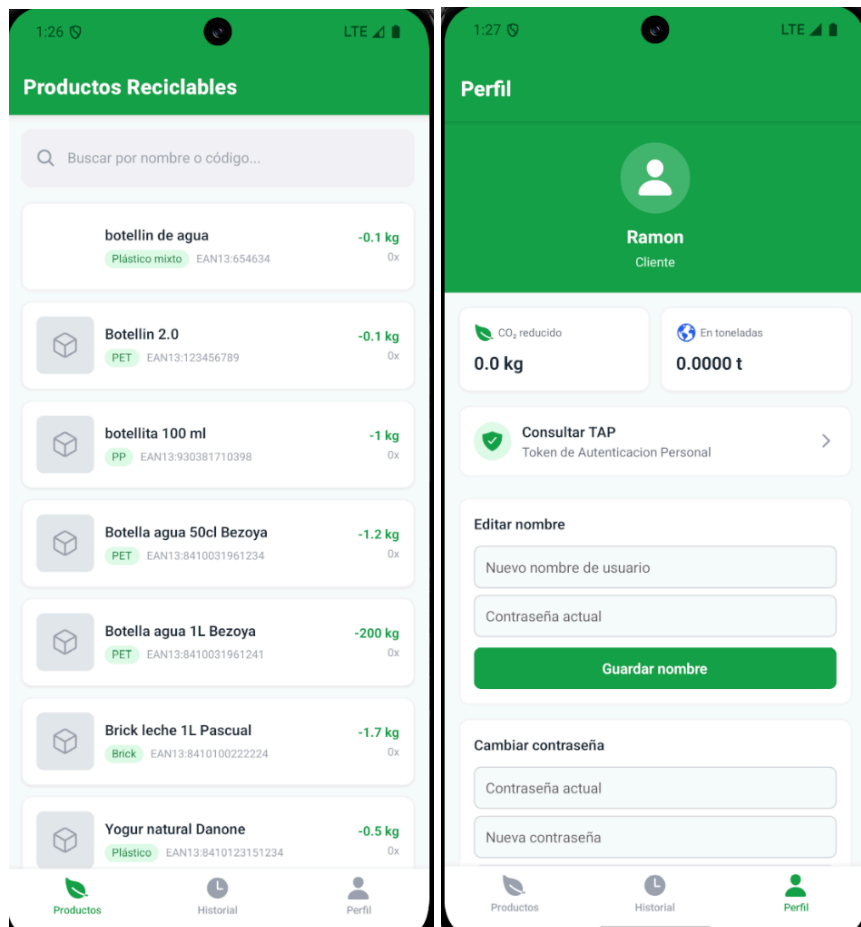
store/

- |— **authStore.ts** <-- Token JWT, usuario, login/logout, modo invitado
- |— **connectionStore.ts** <-- Estado de conexión con la API
- |— **productStore.ts** <-- Caché de productos
- |— **recycleStore.ts** <-- Historial de reciclaje

services/

- |— **api.ts** <-- Interfaz ApiService (contrato)
- |— **api.client.ts** <-- RealApiService (HTTP)
- |— **api.offline.ts** <-- OfflineApiService (datos simulados)
- |— **index.ts** <-- Factory de servicios

Algunas de estas ventanas se ven tal que así:



2.6.5. Terminal de Reciclaje (Python)

Adicionalmente, se ha desarrollado un terminal de reciclaje en Python para puntos físicos. Este componente complementa el sistema permitiendo el registro de reciclajes mediante escaneo de códigos de barras e identificación por TAP.

- Utiliza la librería requests para comunicación HTTP con la API REST.
- Interfaz de terminal con colores ANSI para mejorar la experiencia visual.
- Configuración mediante variables de entorno (RECIAPP_API_URL, RECIAPP_ADMIN_USER, RECIAPP_ADMIN_PASS).

- Flujo de 3 pasos: escanear producto -> introducir TAP -> confirmar reciclaje.

```
RECIAPP - Terminal de Reciclaje

Conectando con la API en http://52.201.91.206:3000...
✓ Conexión con la API establecida.

Autenticando terminal...
✓ Terminal autenticado correctamente.

===== NUEVO RECICLAJE =====

Paso 1/3: Escanea el código de barras del producto
(escribe 'salir' para terminar)

📦 Código de barras: |
```

3. FASE DE PRUEBAS

Se ha diseñado una batería de pruebas funcionales para verificar el correcto funcionamiento de todos los componentes del sistema. Las pruebas se han realizado de forma manual, comprobando cada flujo de la aplicación desde los distintos clientes.

3.1. Pruebas de la API REST

Se verificaron todos los endpoints de la API utilizando Postman, enviando peticiones HTTP reales al servidor desplegado en AWS EC2. Cada prueba comprueba el código de respuesta HTTP, la estructura del JSON devuelto y el correcto funcionamiento de la autenticación JWT. Los resultados se resumen en la siguiente tabla:

Tabla completa (P-01 a P-12):

ID	Prueba	Entrada	Resultado esperado	Estado
P-01	Health check	GET /api/health	{"status":"ok"}	OK
P-02	Registro de usuario	POST /api/usuarios/registro con nombre y contraseña	201 Created + token JWT	OK
P-03	Registro duplicado	POST /api/usuarios/registro con nombre existente	409 Conflict	OK
P-04	Login correcto	POST /api/usuarios/login con credenciales válidas	200 OK + token JWT	OK

P-05	Login incorrecto	POST /api/usuarios/login con contraseña errónea	401 Unauthorized	OK
P-06	Acceso sin token	GET /api/productos sin cabecera Authorization	401 Unauthorized	OK
P-07	Acceso con token	GET /api/productos con Bearer token válido	200 OK + lista de productos	OK
P-08	Solicitar TAP	PUT /api/usuarios/{id}/tap	200 OK + TAP de 6 dígitos	OK
P-09	Buscar por TAP	GET /api/usuarios/by-tap/{tap}	200 OK + datos del usuario	OK
P-10	Registrar reciclaje	POST /api/historial con idUsuario, tipo y numeroBarras	201 Created + emisiones acumuladas	OK
P-11	Admin sin permisos	GET /api/admin/usuarios con token de cliente	403 Forbidden	OK
P-12	Admin con permisos	GET /api/admin/usuarios con token de admin	200 OK + lista de usuarios	OK

3.2. Pruebas de la aplicación móvil

Las pruebas de la aplicación móvil se realizaron manualmente en un dispositivo físico Android mediante Expo Go y en un emulador Android Studio. Se verificó cada pantalla, la navegación entre pestañas, la respuesta a errores y el modo offline. Los resultados se detallan a continuación:

ID	Prueba	Procedimiento	Resultado esperado	Estado
M-01	Login exitoso	Introducir credenciales válidas y pulsar Iniciar Sesión	Redirección a pantalla de Productos	OK
M-02	Login fallido	Introducir contraseña incorrecta	Alert con mensaje de error	OK
M-03	Registro	Rellenar formulario de registro con datos nuevos	Cuenta creada y redirección a Productos	OK
M-04	Ver productos	Navegar a pestaña Productos	Lista de productos cargada desde la API	OK
M-05	Filtrar por material	Pulsar chip de material (Aluminio)	Solo se muestran productos de Aluminio	OK
M-06	Buscar producto	Escribir nombre en la barra de búsqueda	Lista filtrada en tiempo real	OK
M-07	Ver historial	Navegar a pestaña Historial	Lista de reciclajes con total de CO2	OK
M-08	Ver/solicitar TAP	Ir a Perfil → Ver TAP → Solicitar TAP	Modal muestra TAP de 6 dígitos	OK

M-09	Cambiar nombre	Perfil → nuevo nombre + contraseña actual	Nombre actualizado correctamente	OK
M-10	Modo offline	Abrir app sin conexión → Modo offline	App funciona con datos de ejemplo	OK
M-11	Pull to refresh	Deslizar hacia abajo en Productos	Lista se recarga desde la API	OK

3.3. Pruebas de la aplicación de escritorio

La aplicación de escritorio se probó ejecutándola con JDK 24 y JavaFX 25 en Windows. Se verificaron las funcionalidades de login, navegación entre pestañas, operaciones CRUD de administración y la correcta carga de datos desde la API. Las pruebas realizadas fueron las siguientes:

ID	Prueba	Procedimiento	Resultado esperado	Estado
E-01	Login	Introducir credenciales en la ventana de login	Acceso al dashboard principal	OK
E-02	Tab Admin visible	Login con cuenta de administrador	Pestaña de administración visible	OK
E-03	Tab Admin oculta	Login con cuenta de cliente	Pestaña de administración no visible	OK

E-04	CRUD Usuarios	Crear, modificar y eliminar un usuario de prueba	Operaciones reflejadas en la tabla	OK
E-05	CRUD Productos	Crear, modificar y eliminar un producto de prueba	Operaciones reflejadas en la tabla	OK

3.4. Pruebas del terminal Python

El terminal de reciclaje se probó conectándolo al servidor de producción en EC2. Se verificó la conexión con la API, la autenticación del operador, el escaneo de códigos de barras, la búsqueda de usuarios por TAP y el registro de transacciones de reciclaje:

ID	Prueba	Procedimiento	Resultado esperado	Estado
T-01	Conexión API	Ejecutar terminal con API activa	Mensaje: Conexión con la API establecida	OK
T-02	Login operador	Autenticar con credenciales de admin	Mensaje: Terminal autenticado correctamente	OK
T-03	Escanear producto	Introducir código 8410076472885	Producto Coca-Cola 330ml encontrado	OK
T-04	Producto inexistente	Introducir código inventado	Error: Producto no encontrado	OK

T-05	TAP válido	Introducir TAP de un usuario existente	Usuario verificado con nombre y emisiones	OK
T-06	TAP inválido	Introducir TAP inexistente	Error: No se encontró usuario con ese TAP	OK
T-07	Reciclaje completo	Escanear + TAP + confirmar	Reciclaje registrado, emisiones acumuladas mostradas	OK

3.5. Pruebas de integración

Las pruebas de integración verifican que los distintos componentes del sistema (móvil, escritorio, terminal, API) interactúan directamente entre sí, compartiendo datos en tiempo real a través de la base de datos centralizada:

ID	Prueba	Procedimiento	Resultado esperado	Estado
I-01	Terminal → Móvil	Registrar reciclaje en terminal y verificar en app móvil	El historial de la app móvil refleja el nuevo reciclaje	OK
I-02	Escritorio → Móvil	Crear producto en escritorio y verificar en app móvil	El nuevo producto aparece en la lista de productos móvil	OK
I-03	Móvil → Terminal	Solicitar TAP en app móvil y usarlo en terminal	El terminal reconoce el TAP	OK

			y permite reciclar	
I-04	Concurr encia	Usar app móvil y escritorio simultáneamente	Ambas apps funcionan sin interferencias	OK

4. DOCUMENTACIÓN DE LA APLICACIÓN

4.1. Introducción a la aplicación

ReciApp es un sistema de gestión de reciclaje multiplataforma que permite registrar productos reciclados y realizar un seguimiento de las emisiones de CO2 reducidas por cada usuario. El sistema está compuesto por cuatro aplicaciones que se comunican con una API REST centralizada.

El sistema está pensado para dos tipos de usuarios: clientes (usuarios estándar que reciclan y consultan su impacto) y administradores (que gestionan el catálogo de productos, los usuarios y las transacciones). Adicionalmente, la app móvil permite un modo invitado para explorar la aplicación sin necesidad de registro.

4.2. Manual de instalación

4.2.1. Requisitos previos

Componente	Requisito
Base de datos	MySQL 8.x instalado y en ejecución
API REST	JDK 21 (Eclipse Temurin recomendado)
App escritorio	JDK 24, JavaFX 25
App móvil	Node.js 20 LTS, Expo CLI, dispositivo Android/iOS o emulador
Terminal	Python 3.10+, librería requests

4.2.2. Instalación de la base de datos

Ejecutar los siguientes comandos en el servidor MySQL:

```
mysql -u root -p < sql/000_create_database.sql
```

```
mysql -u root -p < sql/001_imagen_longtext.sql
```

```
# Los sql se encuentran en la carpeta  
.\ReciApp_Final\API\source\sql
```

4.2.3. Instalación de la API REST

```
cd .\ReciApp_Final\API\source
```

```
./gradlew build
```

```
# Compilar el proyecto
```

```
java -jar build/libs/api-0.0.1-SNAPSHOT.jar
```

```
# Ejecutar en puerto 3000
```

La API escuchara en `http://0.0.0.0:3000`. Se debe verificar que el puerto 3000 esté abierto en el firewall o Security Group de AWS.

4.2.4. Instalación de la app móvil

```
cd .\ReciApp_Final\App_Movil\source
```

```
npm install # Instalar dependencias
```

```
npx expo start # Iniciar servidor de desarrollo
```

Para desarrollo, se usa `expo start` y el emulador. Para distribución, se genera un APK nativo mediante `expo prebuild` (que crea el proyecto Android nativo) seguido de `gradlew assembleRelease`. La compilación requiere el Android NDK 27 y JDK 17. El APK resultante (~50 MB, con arquitecturas `arm64-v8a` y `x86_64`) se puede instalar directamente en cualquier dispositivo Android o emulador. El `AndroidManifest` incluye `usesCleartextTraffic=true` para permitir conexiones HTTP al backend.

- Nota: El APK usa la clave de depuración. Para Google Play se necesita una keystore propia.

- APK generado en:
`android/app/build/outputs/apk/release/app-release.apk`

- Comando: `cd android && gradlew.bat assembleRelease`

- Requisitos de compilacion: Android SDK, NDK 27.1.12297006, CMake 3.22.1, JDK 17.

4.2.5. Instalación de la app de escritorio

```
cd .\ReciApp_Final\App_Escritorio\source
```

```
./gradle run
```

La aplicación de escritorio se distribuye como un instalador gráfico para Windows (.exe con wizard de instalación y .msi para despliegue silencioso). Se utiliza el plugin badass-runtime-plugin (org.beryx.runtime v1.13.1) en Gradle, que invoca jpackage del JDK 24. jpackage delega en WiX Toolset 3.14 para compilar los instaladores Windows. El resultado incluye un JRE embebido reducido, por lo que el usuario no necesita instalar Java.

- Requisitos de compilacion: JDK 24, JavaFX 25, Gradle 8.8, WiX Toolset 3.14, plugin org.beryx.runtime 1.13.1.

- El instalador EXE (~100 MB) permite elegir directorio, crea acceso directo en escritorio y entrada en menú Inicio. El MSI (~99 MB) permite instalación silenciosa via msiexec /quiet.

- Alternativamente, sin WiX, se puede generar un ZIP portable (~98 MB) con gradlew.bat :app:runtime.

- Comando para generar el instalador: gradlew.bat :app:jpackage (requiere WiX en el PATH del sistema).

4.2.6. Instalación del terminal Python

```
cd terminal
```

```
pip install requests # Instalar dependencia
```

```
python reciclaje_terminal.py # Ejecutar
```

Opcionalmente, se pueden configurar variables de entorno: RECIAPP_API_URL, RECIAPP_ADMIN_USER y RECIAPP_ADMIN_PASS.

4.2.7. Despliegue con Docker

```
# docker-compose.yml
version: '3.8'
services:
  mysql:
    image: mysql:8.0
    container_name: reciapp-db
    environment:
      MYSQL_ROOT_PASSWORD: clase1234
      MYSQL_DATABASE: reciInventario_db
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
      -
      ./sql/000_create_database.sql:/docker-entrypoint-initdb.d/01.sql
      -
      ./sql/001_imagen_longtext.sql:/docker-entrypoint-initdb.d/02.sql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      retries: 5

  api:
    build: ./api-spring
    container_name: reciapp-api
    ports:
      - "3000:3000"
    environment:
      SPRING_DATASOURCE_URL:
jdbc:mysql://mysql:3306/reciInventario_db
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: clase1234
    depends_on:
      mysql:
        condition: service_healthy

volumes:
  mysql_data:
```

```
# api-spring/Dockerfile
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY build/libs/*.jar app.jar
EXPOSE 3000
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Para levantar todo el sistema con Docker:
docker compose up -d

4.3. Manual de usuario

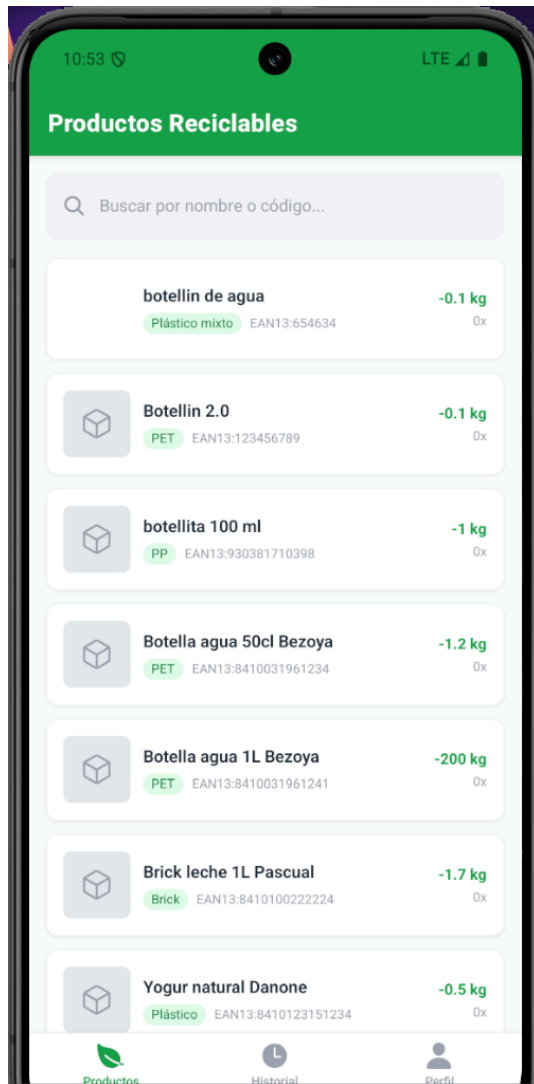
4.3.1. Registro e inicio de sesión

Al abrir la aplicación (móvil o escritorio), se muestra la pantalla de inicio de sesión. Si el usuario no tiene cuenta, puede pulsar en 'Regístrate' para acceder al formulario de registro. Se debe introducir un nombre de usuario único y una contraseña. Una vez registrado, el sistema inicia sesión automáticamente.



4.3.2. Consultar productos

La pestaña 'Productos' muestra el catálogo completo de productos reciclables. Cada tarjeta muestra el nombre del producto, su material, código de barras y la cantidad de CO2 que se reduce al reciclarlo. Se puede buscar por nombre o código de barras usando la barra de búsqueda, y filtrar por tipo de material usando los chips superiores.



4.3.3. Consultar historial de reciclaje

La pestaña 'Historial' muestra todas las acciones de reciclaje realizadas por el usuario. En la cabecera se visualiza el total de CO2 reducido acumulado. Cada entrada del historial incluye el nombre del producto, su material, la fecha del reciclaje y las emisiones reducidas.

4.3.4. Gestionar perfil

La pestaña 'Perfil' permite al usuario ver sus estadísticas (emisiones en kg y toneladas), consultar y solicitar su TAP, cambiar su nombre de usuario o contraseña, y eliminar su cuenta.

4.3.5. Uso del TAP en el terminal de reciclaje

Para reciclar en un punto físico, el usuario debe tener un TAP activo (se solicita desde el perfil de la app móvil). En el terminal de reciclaje, el operador escanea el código de barras del producto y el usuario introduce su TAP de 6 dígitos. El sistema registra el reciclaje y muestra la confirmación con las emisiones acumuladas.

4.3.6. Modo offline (app móvil)

Si la app móvil no puede conectar con la API, se muestra un modal con dos opciones: 'Recargar' para reintentar la conexión, o 'Modo offline' para explorar la app con datos de ejemplo. En modo offline, las funcionalidades de autenticación y gestión están deshabilitadas.

4.4. Manual de administración

La administración del sistema se realiza principalmente desde la aplicación de escritorio (JavaFX), que ofrece una interfaz gráfica completa para la gestión de usuarios, productos y transacciones.

4.4.1. Acceso al panel de administración

Al iniciar sesión con una cuenta de administrador, el dashboard muestra una pestaña adicional de 'Administración' que no está visible para los usuarios con rol cliente.

4.4.2. Gestion de usuarios

La sección de usuarios muestra una tabla con todos los usuarios del sistema (ID, nombre, permisos, emisiones, TAP). El administrador puede crear nuevos usuarios, modificar sus datos (incluyendo cambiar su rol) y eliminar usuarios. La eliminación es en cascada: se borran automáticamente los registros de reciclaje asociados.

4.4.3. Gestión de productos

La sección de productos muestra el catálogo completo. El administrador puede crear nuevos productos indicando tipo de código, número de barras, nombre, material, emisiones reducibles e imagen (en Base64). También puede modificar cualquier campo de un producto existente o eliminarlo.

4.4.4. Gestión de transacciones

La sección de transacciones muestra el historial de reciclaje de todos los usuarios. El administrador puede crear transacciones manualmente (útil para corregir datos o registrar reciclajes retroactivos) y eliminar transacciones incorrectas.

5. CONCLUSIONES FINALES

5.1. Grado de cumplimiento de los objetivos

Se ha logrado desarrollar un sistema multiplataforma completo y funcional que cumple con todos los objetivos planteados al inicio del proyecto:

- Se ha implementado una API REST robusta con Spring Boot que gestiona toda la lógica de negocio, la autenticación JWT y la persistencia en MySQL.

- Se ha desarrollado una aplicación móvil multiplataforma con React Native y Expo que permite a los usuarios consultar productos, ver su historial y gestionar su perfil.

- Se ha creado una aplicación de escritorio con JavaFX que proporciona funcionalidades de administración completas.

- Se ha implementado un terminal de reciclaje en Python para puntos físicos, utilizando códigos de barras y TAP como método de identificación.

- El sistema se ha desplegado exitosamente en un servidor AWS EC2, demostrando su viabilidad en un entorno real.

- El proyecto cubre los módulos de Acceso a Datos, Desarrollo de Interfaces, Programación de Servicios y Procesos, y Programación Multimedia y Dispositivos Móviles.

5.2. Propuesta de modificaciones y ampliaciones futuras

A continuación se proponen posibles mejoras y ampliaciones del sistema para futuras versiones:

- Sistema de rankings y gamificación: implementar tablas de clasificación de usuarios por emisiones reducidas, insignias y logros.

- Notificaciones push: avisar a los usuarios de nuevos productos añadidos al catálogo o recordatorios de reciclaje.

- Escaneo de códigos de barras desde la app móvil: integrar la cámara del dispositivo para escanear productos directamente, sin necesidad de un terminal físico.
- Dashboard web de administración: crear un panel de control web con React o Angular que complemente la app de escritorio.
- Análisis de datos: implementar estadísticas avanzadas con gráficos temporales, comparativas por materiales y predicciones de impacto.
- Internacionalización (i18n): traducir la aplicación a otros idiomas para ampliar su alcance.
- Autenticación con redes sociales: permitir login con Google, Apple o Facebook para facilitar el registro.
- Tests automatizados: implementar tests unitarios (JUnit, Jest) y de integración (Cypress, Detox) para mejorar la calidad del código.

6. BIBLIOGRAFÍA

Documentación oficial y recursos utilizados durante el desarrollo del proyecto:

Backend y base de datos

- Spring Boot Reference Documentation.
<https://docs.spring.io/spring-boot/docs/current/reference/html/>
- Spring Security Reference.
<https://docs.spring.io/spring-security/reference/index.html>
- Spring Data JPA Reference.
<https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
- MySQL 8.0 Reference Manual.
<https://dev.mysql.com/doc/refman/8.0/en/>
- JSON Web Tokens (JWT) Introduction.
<https://jwt.io/introduction>
- JJWT - Java JWT. <https://github.com/jwtk/jjwt>

Aplicación móvil

- React Native Documentation.
<https://reactnative.dev/docs/getting-started>
- Expo Documentation. <https://docs.expo.dev/>
- Expo Router Documentation.
<https://docs.expo.dev/router/introduction/>
- NativeWind Documentation. <https://www.nativewind.dev/>
- Zustand - State Management.
<https://docs.pmnd.rs/zustand/getting-started/introduction>
- React Native Paper.
<https://callstack.github.io/react-native-paper/>

Aplicación de escritorio

- OpenJFX Documentation. <https://openjfx.io/javadoc/21/>

- JavaFX CSS Reference Guide.

<https://openjfx.io/javadoc/21/javafx.graphics/javafx/scene/doc-files/cssref.html>

- Gson User Guide.

<https://github.com/google/gson/blob/master/UserGuide.md>

Terminal y Python

- Python Requests Library.

<https://docs.python-requests.org/en/latest/>

- Python 3 Documentation. <https://docs.python.org/3/>

Despliegue e infraestructura

- AWS EC2 User Guide. <https://docs.aws.amazon.com/ec2/>

- Docker Documentation. <https://docs.docker.com/>

Herramientas de desarrollo

- Git Documentation. <https://git-scm.com/doc>

- Postman Learning Center.

<https://learning.postman.com/docs/getting-started/introduction/>

- IntelliJ IDEA Documentation.

<https://www.jetbrains.com/idea/documentation/>

7. AGRADECIMIENTOS

Esta parte ya no es relevante para el trabajo pero quería ponerla para destacar las ayudas de personas que he recibido durante este curso

Quería agradecer la labor de Juan Jesus Navarro que tanto para mi como para todos mis compañeros ha sido un delegado excelente, además de esto a nivel más personal me ha ayudado con trabajos y me ha permitido a hacer tareas que no habría tenido posibilidad de hacer por el tiempo que tardaría en volver a mi casa.

También quiero agradecer la ayuda que he recibido por parte de mi pareja, que me ha permitido estar días prácticamente enfrente del ordenador solucionando problemas y seguir aún más por la noche, todo gracias a que se ha estado esforzando muchísimo para que durante esos días de estrés pudiera no tener ninguna otra distracción y poder seguir estudiando.